

Code Conventies 2025

10 DECEMBER

Gemaakt door: Narek Wanis, Bram Daalmans,
Dani Hugens, Aleksandar Atanasov.



Inhoud

Code Conventies	3
------------------------------	----------

Code Conventies

Laravel Code Conventies (NL)

Doel

- Biedt consistente regels en voorbeelden voor het schrijven van Laravel-applicaties.
- Vergemakkelijkt samenwerking, review en onderhoudbaarheid.

Scope

- Gilt voor PHP/Laravel code, Blade-templates, tests, migrations, seeds, artisan commands en front-end assets die in een Laravel-project leven.

Referenties

- Volg PSR-12 voor PHP-codering als basis.
- Laravel documentatie en best practices: <https://laravel.com/docs>

Algemene regels

- Volg PSR-12: gebruik type hints, return types waar mogelijk, en PSR-4 autoloading.
- PHP versie: gebruik minimaal de versie die je project vereist; geef voorkeur aan stabiele LTS-versies.
- Bestandsnamen en klassen: PSR-4 PascalCase voor klassen en controller-namen; bestandsnaam = classnaam.
- Gebruik 4 spaties voor indentatie (geen tabs).
- Max lange lijnen: streef naar 120 tekens; breek expressies indien nodig.
- Groepeer use-statements en sorteert alfabetisch per group (builtin, vendor, app).
- Laat 1 lege regel tussen methodes; 2 lege regels tussen classes.

Structuur en naamgeving

- Controllers: App\Http\Controllers, naam eindigt op Controller, bijv. UserController.
- Models: App\Models, PascalCase (User, OrderItem).
- Repositories/Services: App\Services of App\Repositories, PascalCase met suffix Service/Repository.
- Requests: App\Http\Requests, naam eindigt op Request, bijv. StoreUserRequest.
- Resources: App\Http\Resources, naam eindigt op Resource, bijv. UserResource.
- Policies: App\Policies, naam eindigt op Policy.

-
- Events/Listeners: App\Events, App\Listeners, benoem volgens domein (OrderPlaced, SendOrderNotification).
 - Jobs: App\Jobs, asynchrone taken, naam eindigt op Job.
 - Tests: tests/Feature en tests/Unit, testmethoden met duidelijke namen: test_can_create_user_with_valid_data() of public function testUserCreation().

Classes en methods

- Visibility expliciet (public/protected/private).
- Gebruik dependency injection in constructors/methoden in plaats van facades wanneer mogelijk.
- Keep controllers thin: zet businesslogica in Services, Actions of Model-methoden.
 - Eén verantwoordelijkheid per class/methode.
 - Methoden: klein, max ~50-80 regels idealiter.

Type hints & PHPDoc

- Gebruik type hints voor parameters en return types.
- Voeg PHPDoc toe waar types niet express in code kunnen (bijv. complexe arrays), of voor public API van packages.
- Docblocks voor classes beschrijven doel en eventuele belangrijke notities.

Eloquent & Models

- Gebruik guarded/fillable expliciet. Voorkeur: protected \$fillable = [...]
- Gebruik casts: protected \$casts = ['is_active' => 'boolean', 'price' => 'decimal:2'];
- Eloquent-relaties benoemen in enkelvoud/plural passend bij relation: public function user(), public function orderItems().
 - Gebruik query scopes: scopeActive(\$query) → gebruik ->active() in queries.
- Mutators / Accessors: gebruik moderne Attribute classes (Laravel 9+) wanneer mogelijk.

Voorbeeld (modern):

```
```php
protected function title(): Attribute
{
 return Attribute::make(
 get: fn ($value) => strtoupper($value),
 set: fn ($value) => trim($value),
);
}
```

---

```
}
...
```

- Gebruik route model binding waar kan: `Route::get('/users/{user}', [UserController::class, 'show']);`

### Migrations & Database

- Naamgeving: `timestamp_create_table_name_table.php` en foreign keys: `table_column_foreign`
- Gebruik `Schema::create/Schema::table` en `up/down` (of `use ->after()` indien vereist).
  - Voeg indexes en foreign keys expliciet toe:

```
```php  
$table->unsignedBigInteger('user_id');  
$table->foreign('user_id')->references('id')->on('users')->onDelete('cascade');  
...`
```
- Gebruik nullable waar nodig, en default waarden expliciet.
- Prefer integer unsigned voor id foreign keys (`BigInteger` als primary ids `BigIncrements`).

Validation

- Gebruik Form Requests voor gevalideerde logic en autorisatie (`StoreUserRequest`).
 - Haal validatie- en authorize logica uit controllers.
- Gebruik valide regels en custom regels (`Rule::unique(...)`, `Rule::in(...)`).

Controllers

- Keep slim; behandel autorisatie (`authorize`) en validatie (`FormRequest`) en roep services aan.
- RESTful resource controllers en API resources voor responses.

Voorbeeld:

```
```php  
public function store(StorePostRequest $request, PostService $service)
{
 $post = $service->create($request->validated());
 return new PostResource($post);
}
...`
```

- Gebruik type-hinting voor modellen via route-model binding.

---

## Routes

- Organiseer routes in routes/web.php en routes/api.php.
- Groepeer middleware en prefix: `Route::middleware('auth')->prefix('admin')->group(...)`.
- Gebruik named routes: `->name('admin.users.index')`.
- Vermijd logica in routes; route moet alleen naar controller/action verwijzen.

## Responses & API

- Gebruik API Resources (`App\Http\Resources`) voor consistente JSON structuren.
- HTTP status codes correct gebruiken (201 Created, 204 No Content, 400 Bad Request, 422 Validation errors).
- Hanteer consistent error response format (status, message, errors).

## Exception handling & Logging

- Gebruik eigen exceptions voor domein-fouten en behandel in Handler.php indien speciale respons nodig.
- Gebruik `Log::channel('slack')->error(...)` met context arrays; geef geen gevoelige informatie.

## Security

- Nooit vertrouw user input; valideer en ontsmet.
- Gebruik prepared statements / Eloquent preventing SQL injection.
  - Hash wachtwoorden met `Hash::make()`.
  - Gebruik gates/policies voor autorisatie.
- Configuratie: geheimen in .env, niet committen; gebruik config caching in productie (`php artisan config:cache`).

## Blade templates

- Componenten in resources/views/components, gebruik kebab-case component names: `<x-alert />`
- Escaping: `{{ $var }}` voor escaped output, `{!! $html !!}` alleen als veilig.
  - Minimaliseer logica in views; gebruik presentors/Resources.
  - Gebruik localization helpers `__('message.key')`.

## Front-end assets

- Gebruik Laravel Mix/Vite volgens projectconfiguratie.

- 
- Organiseer JS in resources/js, CSS in resources/css.
  - Component-specifieke JS minimaliseren; prefer modular import.

### Testing

- Volg Arrange-Act-Assert.
- Gebruik model factories, RefreshDatabase of DatabaseTransactions.
- Schrijf feature tests voor HTTP endpoints en unit tests voor services.
- Naam tests duidelijk: test\_guest\_cannot\_access\_admin\_routes().

### Queues, Jobs en Events

- Lange taken in Jobs, synchronous logica vermijden.
  - Events en Listeners voor losse koppeling.
  - Retry logica en failed jobs configureren.

### Formatting & Git

- Gebruik een pre-commit hook (e.g. husky, laravel pint) om formatting te controleren.
  - Gebruik Laravel Pint of PHP-CS-Fixer met PSR-12 regels.
- Commit messages: korter samenvattend onderwerp + optionele body. Maak commits atomisch.

### Voorbeelden

#### Model (User):

```
```php
<?php
namespace App\Models;

use Illuminate\Database\Eloquent\Model;
use Illuminate\Database\Eloquent\Casts\Attribute;

class User extends Model
{
    protected $fillable = ['name', 'email', 'password'];
    protected $casts = [
        'email_verified_at' => 'datetime',
        'is_admin' => 'boolean',
    ];
}
```

```

        ];

        protected function name(): Attribute
        {
            return Attribute::make(
                get: fn ($value) => ucfirst($value),
                set: fn ($value) => trim($value)
            );
        }

        public function posts()
        {
            return $this->hasMany(Post::class);
        }
    }
    ...

    Controller (resource):
    ```php
 <?php
 namespace App\Http\Controllers;

 use App\Http\Requests\StorePostRequest;
 use App\Http\Resources\PostResource;
 use App\Services\PostService;

 class PostController extends Controller
 {
 public function store(StorePostRequest $request, PostService $service)
 {
 $post = $service->create($request->validated());
 return new PostResource($post);
 }
 }
 ...

```



---

```
FormRequest:
```php
<?php
namespace App\Http\Requests;

use Illuminate\Foundation\Http\FormRequest;

class StorePostRequest extends FormRequest
{
    public function authorize()
    {
        return $this->user()->can('create', Post::class);
    }

    public function rules()
    {
        return [
            'title' => 'required|string|max:255',
            'body' => 'required|string',
        ];
    }
}
```

```
Migration:
```php
public function up()
{
 Schema::create('posts', function (Blueprint $table) {
 $table->id();
 $table->foreignId('user_id')->constrained()->onDelete('cascade');
 $table->string('title');
 $table->text('body');
 $table->timestamps();
 });
}
```

---

...

Blade component (alert):

```
``blade
<div {{ $attributes->merge(['class' => 'alert alert-'. $type]) }}>
 {{ $slot }}
</div>
...

```

Best practices checklist voor code review

- Volgt de code PSR-12 / Pint regels?
- Zijn controllers skinny en services/repositories gebruikt voor business logic?
  - Zijn FormRequests gebruikt voor validatie?
  - Is autorisatie correct toegepast (gates/policies)?
  - Worden API Resources gebruikt voor output?
- Zijn tests aanwezig voor nieuwe/gewijzigde functionaliteit?
  - Geen gevoelige data in logs of errors?
  - Security checks: SQL injection, XSS, CSRF?

Aanpassing voor jouw project

- Pas regels aan voor legacy code of specifieke teamafspraken (bijv. tab-size, max line length).
- Voeg extra secties toe voor packages, microservices, of CI/CD pipelines.

Bijlagen & toolsuggesties

- Gebruik Laravel Pint voor code formatting.
- Gebruik PHPStan / Psalm voor statische analyse.
  - Gebruik Pest of PHPUnit voor tests.

Einde document

- Dit is een generiek, aanpasbaar startpunt. Voor project-specifieke conventies kan ik het document aanpassen en uitbreiden (bijv. naming conventions voor events, specifieke services, of CI checks).